Nombre: Deury Dayron De La Cruz Ubiera

Matricula: 2023-2022

Materia: Programación **II**

Fecha: 25/03/2026

Tema: Sad

Profesor: Francis Ramírez

Indice:

1. Introducción.....5
Propósito.....5
Alcance.....5
Definiciones y Acrónimos.....5
Tecnologias y decisiones de proyecto.....6
.....6
Visión General del Documento.....6
2. Vista General del Sistema.....8
Descripción General.....8
Principios Arquitectónicos.....9
Componentes Principales.....9
3. Vista Lógica.....9
Descripción.....9
Módulos del Dominio.....10
Módulo de Usuarios y Roles.....10
Módulo de Recursos Bibliográficos.....10
Módulo de Categorías.....10
Módulo de Préstamos.....10
Módulo de Penalizaciones.....10
Módulo de Auditoría.....11
Objetos de Valor.....11
Servicios de Dominio.....11
4. Vista de Procesos.....12
Descripción.....12
Proceso de Creación de Préstamo.....12
Proceso de Devolución.....12
Flujo de Autenticación.....12
5. Vista de Desarrollo.....13
Descripción.....13
Estructura en Capas.....13
Capa de Dominio.....13
Capa de Aplicación.....13
Capa de Infraestructura.....13
Capa de Presentación (APIs).....13
Flujo de Dependencias entre Capas.....14
Patrón de Diseño de Entidades.....14
6. Vista de Despliegue.....15
Descripción.....15
Nodos de Despliegue.....15
Comunicación entre Nodos.....15
Puertos de Acceso.....15
7. Vista de Casos de Uso.....16
Descripción.....16
Caso de Uso Arquitectónicamente Significativo: Registrar Préstamo.....16
Caso de Uso Arquitectónicamente Significativo: Consultar Recursos Disponibles....16

8. Decisiones Arquitectónicas....17
Adopción de Domain-Driven Design....17
Patrón Aggregate Root para Préstamo....17
Patrón Result para Manejo de Errores de Negocio....17
Dos APIs Separadas según Audiencia....18
Herencia para Tipos de Recursos....18
Inyección de Dependencias como Mecanismo de Desacoplamiento....18
Auditoría Automática en Operaciones Relevantes....19

1. Introducción

Propósito

Este documento describe la arquitectura de software del Sistema de Gestión de Biblioteca (SIGEBI). Su objetivo es comunicar las decisiones arquitectónicas tomadas durante el diseño del sistema, explicar la estructura de los componentes y sus relaciones, y servir como referencia técnica para el equipo de desarrollo, arquitectos, testers y cualquier parte interesada en entender cómo está construido el sistema.

El documento sigue el modelo de vistas arquitectónicas 4+1, que permite describir la arquitectura desde múltiples perspectivas para cubrir las necesidades de los diferentes actores involucrados en el proyecto.

Alcance

Este SAD cubre la arquitectura completa de SIGEBI, incluyendo sus cinco componentes principales: la capa de dominio, la capa de aplicación, la capa de infraestructura y las dos interfaces de acceso al sistema (una para el personal administrativo y otra para los usuarios finales). El documento describe la estructura lógica, el flujo de procesos, la organización del código, el despliegue del sistema y los principales casos de uso desde la perspectiva arquitectónica.

Definiciones y Acrónimos

SAD: Software Architecture Document, Documento de Arquitectura de Software.

DDD: Domain-Driven Design, diseño orientado al dominio de negocio.

Capa de Dominio: Núcleo del sistema que contiene las reglas y lógica de negocio.

Capa de Aplicación: Coordina los casos de uso orquestando el dominio y la infraestructura.

Capa de Infraestructura: Implementa el acceso a datos y servicios externos.

API REST: Interfaz de programación de aplicaciones basada en el estilo arquitectónico REST.

Aggregate Root: Entidad principal de un agregado que controla el acceso a sus componentes internos.

Value Object: Objeto sin identidad propia, definido únicamente por sus atributos.

Domain Service: Servicio que encapsula lógica de negocio que no pertenece a una entidad específica.

Repository: Patrón que abstrae el acceso a los datos de persistencia.

ISP: Interface Segregation Principle, principio de segregación de interfaces.

Tecnologias y decisiones de proyecto

C# .NET 8.0

1. PostgreSQL
2. Npgsql
3. BCrypt.Net-Next
4. Microsoft.IdentityModel.JsonWebTokens
5. ASP.NET Core Web API
6. Swashbuckle.AspNetCore (Swagger)
7. Domain-Driven Design (DDD)
8. Patrón Repository
9. Patrón Result
10. Inyección de Dependencias
11. Aggregate Root
12. Value Objects
13. Domain Services
14. Visual Studio 2022
15. pgAdmin
16. Swagger UI
17. React (futuro)

Visión General del Documento

El documento está organizado en ocho secciones. La sección 1 presenta la introducción. La sección 2 ofrece una vista general del sistema. La sección 3 presenta la vista lógica. La sección 4 presenta la vista de procesos. La sección 5 presenta la vista de desarrollo. La sección 6 presenta la vista de despliegue. La sección 7 presenta la vista de casos de uso. La sección 8 documenta las decisiones arquitectónicas más importantes.

2. Vista General del Sistema

Descripción General

SIGEBI está construido siguiendo una arquitectura en capas basada en los principios de Domain-Driven Design (DDD). Este enfoque organiza el sistema alrededor del dominio de negocio, colocando las reglas y lógica del negocio en el centro de la solución, independientemente de los detalles técnicos de implementación.

La arquitectura asegura que las reglas del negocio permanezcan aisladas de los detalles de infraestructura, lo que facilita las pruebas, el mantenimiento y la evolución del sistema a lo largo del tiempo.

Principios Arquitectónicos

El diseño de SIGEBI se guía por los siguientes principios fundamentales:

1-Separación de responsabilidades: cada capa tiene una función bien definida y no invade las responsabilidades de las demás.

2-Dependencia hacia el dominio: todas las capas dependen del dominio, pero el dominio no depende de ninguna capa externa.

3-Inyección de dependencias: los componentes no crean sus propias dependencias, sino que las reciben desde el exterior, reduciendo el acoplamiento.

3-Inversión de dependencias: las capas de nivel superior definen interfaces que las capas de nivel inferior implementan.

3-Entidades inteligentes: las entidades del dominio encapsulan su propio comportamiento y validan sus invariantes de negocio.

4-Patrón Result: las operaciones retornan un objeto que indica éxito o fallo, evitando el uso de excepciones para flujos de negocio normales.

Componentes Principales

El sistema está compuesto por cinco componentes principales que forman una solución integrada. El componente de Dominio contiene las entidades, objetos de valor, servicios de dominio e interfaces de repositorios. El componente de Aplicación contiene los casos de uso, objetos de transferencia de datos y servicios de aplicación. El componente de Infraestructura contiene las implementaciones de repositorios y el acceso a la base de datos. El componente API Administrativa expone los servicios para el personal de la biblioteca. El componente API Pública expone los servicios para los usuarios finales de la institución.

3. Vista Lógica

Descripción

La vista lógica describe la estructura funcional del sistema, sus principales módulos y las relaciones entre ellos. Se organiza alrededor de los conceptos del dominio de negocio de la biblioteca.

Módulos del Dominio

El dominio de SIGEBI está compuesto por los siguientes módulos principales:

Módulo de Usuarios y Roles

Gestiona la identidad y los permisos de acceso al sistema. El módulo incluye las entidades Usuario y Rol, con sus respectivas reglas de validación. El Usuario emplea objetos de valor para garantizar la corrección de su nombre completo y su correo electrónico. Las operaciones sobre usuarios incluyen creación, activación, desactivación y actualización de perfil.

Módulo de Recursos Bibliográficos

Gestiona el catálogo de materiales disponibles en la biblioteca. El módulo implementa una jerarquía de herencia donde existe una entidad base que define el comportamiento común a todos los recursos, y entidades especializadas para cada tipo: Libro y Revista. Cada tipo de recurso tiene atributos y comportamiento específico según su naturaleza. Las operaciones principales son: registro de nuevos recursos, actualización de información, ajuste de stock y consulta de disponibilidad.

Módulo de Categorías

Permite clasificar los recursos bibliográficos en categorías temáticas. Una categoría puede estar activa o inactiva, y no puede desactivarse si tiene recursos asociados.

Módulo de Préstamos

Es el módulo central del sistema. La entidad Préstamo actúa como Aggregate Root, lo que significa que controla el acceso y la creación de sus componentes internos (los detalles del

préstamo). El módulo gestiona el ciclo de vida completo del préstamo: creación, seguimiento, devolución y vencimiento. Las reglas de negocio incluyen verificación de penalizaciones activas, control de stock y cálculo automático de multas por retraso.

Módulo de Penalizaciones

Gestiona las sanciones aplicadas a los usuarios por incumplimientos. Las penalizaciones pueden crearse automáticamente al procesar una devolución tardía o manualmente por el Bibliotecario. Una penalización activa bloquea al usuario para realizar nuevos préstamos hasta que sea finalizada.

Módulo de Auditoría

Registra de forma inmutable todas las acciones relevantes del sistema. Cada registro de auditoría incluye el usuario responsable, la descripción de la acción y la fecha y hora exacta. El módulo de auditoría no permite la modificación ni eliminación de registros.

Objetos de Valor

El dominio utiliza objetos de valor para encapsular conceptos simples con reglas de validación propias. El objeto de valor Correo Electrónico encapsula la validación del formato de un correo electrónico y garantiza que siempre se almacene en minúsculas. El objeto de valor Nombre de Persona encapsula el nombre y apellido de un individuo, exponiendo el nombre completo como una propiedad calculada. El objeto de valor Dinero encapsula un monto monetario y su moneda, e incluye operaciones de suma y multiplicación con validación de que el monto no sea negativo.

Servicios de Dominio

Los servicios de dominio encapsulan lógica de negocio que involucra múltiples entidades y no pertenece naturalmente a ninguna de ellas en particular.

El Servicio de Préstamo coordina las operaciones complejas del ciclo de vida de un préstamo: verificación de condiciones, cálculo de multas a razón de RD$50.00 por día de retraso, procesamiento de devoluciones y marcado de préstamos vencidos.

El Servicio de Penalización gestiona la creación y finalización de penalizaciones, así como la verificación de si un usuario tiene sanciones activas que impidan nuevos préstamos.

4. Vista de Procesos

Descripción

La vista de procesos describe los flujos de ejecución del sistema y cómo interactúan los componentes durante la realización de las operaciones más importantes.

Proceso de Creación de Préstamo

Cuando se solicita la creación de un préstamo, el sistema sigue el siguiente flujo de ejecución. Primero, la capa de aplicación recibe la solicitud con los datos del préstamo. Luego verifica si el usuario solicitante tiene alguna penalización activa consultando el repositorio correspondiente. Si existe una penalización activa, el proceso termina con un resultado de error. Si no hay penalización, la capa de aplicación delega al servicio de dominio la creación del préstamo. El servicio de dominio verifica que la fecha límite sea válida y crea la entidad Préstamo. Posteriormente, para cada recurso solicitado, el sistema verifica la disponibilidad de stock y agrega el detalle al préstamo, reduciendo el stock del recurso. Finalmente, los datos se persisten a través de los repositorios y se registra el evento en auditoría.

Proceso de Devolución

El proceso de devolución sigue el siguiente flujo. La capa de aplicación recibe la solicitud de devolución con el identificador del préstamo. Recupera el préstamo con todos sus detalles desde el repositorio. Delega al servicio de dominio el procesamiento de la devolución. El servicio calcula los días de retraso comparando la fecha actual con la fecha límite del préstamo. Si hubo retraso, calcula el monto de la multa (RD$50.00 por día) y crea una penalización automática para el usuario. El stock de cada recurso devuelto es restaurado. El estado del préstamo cambia a Devuelto. Los cambios se persisten y se registra el evento en auditoría.

Flujo de Autenticación

Cuando un usuario intenta iniciar sesión, el sistema recibe las credenciales. Busca al usuario por su correo electrónico en la base de datos. Verifica que el usuario exista y esté activo. Compara la contraseña proporcionada con el valor cifrado almacenado. Si la verificación es exitosa, el sistema genera un token de acceso con la información del usuario y su rol. Ese token se usa en todas las solicitudes posteriores para identificar al usuario y verificar sus permisos.

5. Vista de Desarrollo

Descripción

La vista de desarrollo describe la organización del código fuente del sistema, sus capas, componentes y dependencias entre ellos.

Estructura en Capas

El sistema está organizado en cuatro capas principales con una dirección estricta de dependencias:

Capa de Dominio

Es el núcleo del sistema. No tiene dependencias hacia ninguna otra capa. Contiene todas las entidades del negocio, los objetos de valor, los servicios de dominio, los enumeradores y las interfaces de repositorios. Las entidades están diseñadas para ser inteligentes: encapsulan su propia lógica de negocio y validan sus invariantes mediante un método estático de creación. El constructor de cada entidad es privado para garantizar que siempre se cree en un estado válido.

Capa de Aplicación

Depende únicamente de la capa de Dominio. Contiene los servicios de aplicación que orquestan los casos de uso del sistema. Cada servicio de aplicación coordina las entidades del dominio, los servicios de dominio y los repositorios para ejecutar una operación completa. También contiene los objetos de transferencia de datos (DTO) que definen la estructura de los datos de entrada y salida del sistema, así como las interfaces de los servicios de infraestructura como el cifrado de contraseñas.

Capa de Infraestructura

Depende únicamente de la capa de Dominio. Implementa las interfaces de repositorios definidas en el dominio, realizando las operaciones de persistencia sobre la base de datos. Cada repositorio es responsable de transformar los datos de la base de datos en entidades del dominio y viceversa. También implementa servicios de infraestructura como el cifrado seguro de contraseñas.

Capa de Presentación (APIs)

Depende de las capas de Aplicación e Infraestructura. Está dividida en dos componentes independientes: uno para el personal administrativo y otro para los usuarios finales. Cada componente expone controladores REST que reciben solicitudes HTTP, delegan el procesamiento a los servicios de aplicación y retornan las respuestas al cliente. También contiene la configuración de inyección de dependencias que conecta todas las capas del sistema.

Flujo de Dependencias entre Capas

Dominio in dependencias externas

Aplicación = Dominio

Infraestructura = Dominio

API Administrativa = Aplicación + Infraestructura

API Pública = Aplicación + Infraestructura

Patrón de Diseño de Entidades

Todas las entidades del sistema siguen el mismo patrón de diseño. El constructor es privado para evitar la creación de instancias en estado inválido. Existe un método estático de creación (Crear) que recibe los parámetros necesarios, valida las reglas de negocio y retorna una nueva instancia válida. Las propiedades solo tienen setters de acceso restringido para evitar modificaciones externas no controladas. Los cambios de estado se realizan únicamente a través de métodos públicos con nombre significativo que expresan la intención del negocio.

6. Vista de Despliegue

Descripción

La vista de despliegue describe cómo los componentes del sistema se distribuyen en la infraestructura de hardware y red.

Nodos de Despliegue

El sistema opera sobre tres nodos principales. El primer nodo es el Servidor de Base de Datos, que aloja el motor de base de datos relacional donde se persiste toda la información del sistema. El segundo nodo es el Servidor de Aplicaciones, que aloja ambos componentes API del sistema: el API Administrativa y el API Pública. Cada uno opera en un puerto diferente para mantener su separación lógica. El tercer nodo es el Cliente, que puede ser cualquier dispositivo con un navegador web moderno desde el cual los usuarios acceden al sistema.

Comunicación entre Nodos

Los clientes se comunican con los servidores de aplicaciones a través de peticiones HTTP seguras. Toda la comunicación entre el cliente y el servidor viaja cifrada. Los servidores de aplicaciones se comunican con el servidor de base de datos a través de la red interna mediante el protocolo del motor de base de datos. El servidor de base de datos no está expuesto directamente a los clientes externos.

Puertos de Acceso

El API Administrativa es accesible en el puerto 7185. Este puerto está destinado exclusivamente al personal de la biblioteca. El API Pública es accesible en el puerto 7011. Este puerto está disponible para los usuarios finales de la institución y para la aplicación web de React que los sirve. La base de datos opera en el puerto estándar del motor de base de datos relacional.

7. Vista de Casos de Uso

Descripción

La vista de casos de uso describe las interacciones más significativas entre los actores del sistema y la arquitectura. Sirve para validar que la arquitectura satisface los requisitos funcionales del sistema.

Caso de Uso Arquitectónicamente Significativo: Registrar Préstamo

Este caso de uso es arquitectónicamente significativo porque involucra la coordinación de múltiples componentes del sistema y aplica varias reglas de negocio complejas. El flujo arquitectónico es el siguiente: el controlador del API recibe la solicitud, la delega al servicio de aplicación de préstamos, el cual invoca al servicio de penalizaciones del dominio para verificar si el usuario tiene sanciones activas, luego invoca al servicio de préstamos del dominio para crear el préstamo y agregar los detalles con validación de stock, y finalmente persiste los cambios a través de los repositorios. Este flujo demuestra cómo la arquitectura separa las responsabilidades: el controlador solo coordina la entrada y salida, el servicio de aplicación orquesta el caso de uso completo, el dominio aplica las reglas de negocio y los repositorios se encargan de la persistencia.

Caso de Uso Arquitectónicamente Significativo: Consultar Recursos Disponibles

Este caso de uso demuestra la separación entre las dos interfaces del sistema. Un Estudiante accede al API Pública, que expone solo los endpoints necesarios para consultar recursos y gestionar préstamos. El controlador del API Pública delega al servicio de aplicación de recursos, que consulta el repositorio y retorna solo la información relevante para el usuario final. El API Administrativo, en cambio, expone el mismo módulo de recursos con capacidades adicionales de creación, actualización y ajuste de stock, restringidas al personal de la biblioteca.

8. Decisiones Arquitectónicas

Adopción de Domain-Driven Design

Decisión: Se adoptó DDD como el estilo arquitectónico principal del sistema.

Justificación: El dominio de gestión de biblioteca tiene reglas de negocio complejas e interrelacionadas (préstamos, penalizaciones, stock). DDD permite modelar estas reglas directamente en el código, haciendo que el sistema sea más comprensible y mantenible. Además, aísla las reglas de negocio de los detalles de implementación, facilitando pruebas y cambios futuros.

Consecuencias: La estructura del código refleja el lenguaje del negocio. Hay una curva de aprendizaje inicial mayor, pero el sistema es más fácil de mantener y extender a largo plazo.

Patrón Aggregate Root para Préstamo

Decisión: La entidad Préstamo fue designada como Aggregate Root, controlando el acceso a los detalles del préstamo.

Justificación: Los detalles de un préstamo no tienen existencia independiente fuera de su préstamo. No tiene sentido crear un detalle sin un préstamo padre. El Aggregate Root garantiza que las invariantes del agregado siempre se mantengan válidas.

Consecuencias: Los detalles del préstamo solo pueden crearse a través de la entidad Préstamo. Esto simplifica el modelo pero requiere que el repositorio cargue el préstamo completo con sus detalles cuando sea necesario.

Patrón Result para Manejo de Errores de Negocio

Decisión: Se implementó un patrón Result propio para retornar resultados de las operaciones de negocio.

Justificación: Las excepciones son costosas y están diseñadas para errores inesperados del sistema, no para flujos normales de negocio como 'el usuario tiene penalización activa'. El patrón Result hace los flujos de error explícitos y predecibles.

Consecuencias: Cada operación retorna un objeto que indica éxito o fallo con un mensaje descriptivo. El código es más verboso pero más claro y predecible en su comportamiento.

Dos APIs Separadas según Audiencia

Decisión: Se implementaron dos APIs independientes: una para el personal administrativo y otra para los usuarios finales.

Justificación: Las necesidades y permisos de cada grupo de usuarios son completamente diferentes. Separar las APIs mejora la seguridad (el API pública no expone operaciones de gestión) y la claridad (cada API tiene un propósito bien definido).

Consecuencias: Existe algo de configuración duplicada entre las dos APIs, pero se obtiene una separación clara de responsabilidades y una superficie de exposición reducida para cada audiencia.

Herencia para Tipos de Recursos

Decisión: Se implementó una jerarquía de herencia para los tipos de recursos bibliográficos, con una clase base y tipos especializados para Libro y Revista.

Justificación: Libros y Revistas comparten comportamiento común (stock, disponibilidad, código, título, autor) pero tienen atributos específicos. La herencia permite compartir el comportamiento común y agregar atributos específicos sin duplicar código.

Consecuencias: El mapeo desde la base de datos requiere determinar el tipo del recurso y crear la instancia correcta. La base de datos almacena todos los recursos en una misma tabla con un campo discriminador de tipo.

Inyección de Dependencias como Mecanismo de Desacoplamiento

Decisión: Se usa inyección de dependencias en toda la aplicación para conectar los componentes.

Justificación: La inyección de dependencias permite que las capas de alto nivel dependan de abstracciones (interfaces) y no de implementaciones concretas. Esto facilita las pruebas, el intercambio de implementaciones y reduce el acoplamiento entre componentes.

Consecuencias: Todos los servicios y repositorios se registran en un contenedor de dependencias al iniciar la aplicación. El sistema es más flexible pero requiere configuración explícita de las dependencias.

Auditoría Automática en Operaciones Relevantes

Decisión: La auditoría se registra automáticamente dentro de los servicios de aplicación, sin requerir acción del usuario.

Justificación: La trazabilidad de las operaciones es un requisito del sistema. Hacerla automática garantiza que ninguna operación importante quede sin registro, independientemente de cómo se ejecute.

Consecuencias: Cada operación importante incluye un paso adicional de registro de auditoría. Esto agrega una operación de escritura por cada acción, pero garantiza la trazabilidad completa del sistema.